SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	V.Sriram	**Reg. No.:**	192524037
Section / Batch:	2025	**Date:**	3-08-2026

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floating-point computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

I V.Sriram(192524037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Sriram

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator **Faculty Incharge**

1)

Debugging Signed Binary Subtraction Using 2's Complement
Identification of Error

The subtraction gives incorrect results because:

- Binary conversion is incorrect.
- 2's complement is not generated properly.
- Overflow is not checked.

Debugging Approach

1. Convert both numbers to binary.
2. Find the 2's complement of the subtrahend.
3. Add it to the minuend.
4. Check the carry and overflow bits.
5. Ignore the carry if there is no overflow.

Corrected Solution
For subtraction:

Example:
A = 12 = 00001100
B = 5 = 00000101

2's Complement of B

11111011

Addition

00001100
11111011

00000111
Result = 7

Optimization

- Use correct 2's complement generation.
- Detect overflow automatically.
- Ignore carry only when appropriate.

Testing

Input	Expected Output	Result
12−5	7	Correct
5−12	-7	Correct
-8−5	-13	Correct

2)

Identification of Error
The Carry Look-Ahead Adder produces incorrect output because:

- Generate (G) logic is incorrect.
- Propagate (P) logic is incorrect.
- Carry equations are wrong.

Debugging Approach
Generate
$G=A\times B$
Propagate
$P=A\oplus B$
Carry
$C1=G0+P0C0$

Verify each carry output.

Corrected Solution

Correct equations:

$G_i = A_i \times B_i$

$P_i = A_i \oplus B_i$

$C_{i+1} = G_i + P_i C_i$

Optimization

- Reduce propagation delay.
- Use parallel carry generation.
- Improve addition speed.

Testing

Inputs:

A=1010

B=0101

Expected Output:

1111

Result:

Correct.

3)

Identification of Error

Extra clock cycles occur because:

- Incorrect bit-pair checking.
- Wrong arithmetic shift.
- Incorrect register update.

Debugging Approach

For each iteration:

- Check Q0 and Q−1.
- Perform Add/Subtract.
- Perform Arithmetic Right Shift.
- Update registers.

Corrected Solution

Decision Table

Q0	Q-1	Operation
00		No Operation
01		Add Multiplicand
10		Subtract Multiplicand
11		No Operation

Repeat until all bits are processed.

Optimization

- Skip unnecessary additions.
- Perform arithmetic shifts correctly.
- Reduce execution cycles.

Testing

6 × 3 = 18

-5 × 4 = -20

Results are correct.

4)

Identification of Error

Incorrect quotient occurs due to:

- Wrong partial remainder.
- Incorrect shift operation.
- Wrong quotient bit generation.

Debugging Approach

1. Shift left.
2. Perform subtraction.
3. If remainder is positive:
    - Quotient bit = 1
4. Else
    - Restore remainder
    - Quotient bit = 0

Repeat until all bits are processed.

Optimization

- Reduce unnecessary restoration.
- Use efficient shifting.
- Update quotient correctly.

5)

Identification of Error

Performance decreases because:

- Double precision is used unnecessarily.
- Large floating-point calculations increase execution time.
- Higher memory usage.

Debugging Approach

Check whether double precision is really required.
If single precision provides sufficient accuracy, replace double precision.

Corrected Solution

Use:
Float instead of double when high precision is not required.

Optimization

- Use single precision.
- Reduce floating-point operations.
- Minimize unnecessary conversions.
- Optimize compiler settings.

Testing

Precision	Execution Time	Accuracy
Double	High	Very High
Single	Low	Sufficient

Result:
Execution speed improves while maintaining acceptable accuracy.